Portfolio Daemon — Architecture Analysis

Project: Portfolio Daemon
Technology: Python, asyncio, asyncssh, httpx, watchdog
Location: `/home/dominic/Documents/portfolio-daemon`
Report #: 2/20

Architecture Type

Background service manager daemon

Layers

CLI/Service Manager → Process Manager → SSH Tunnel → Remote Services

Design Patterns

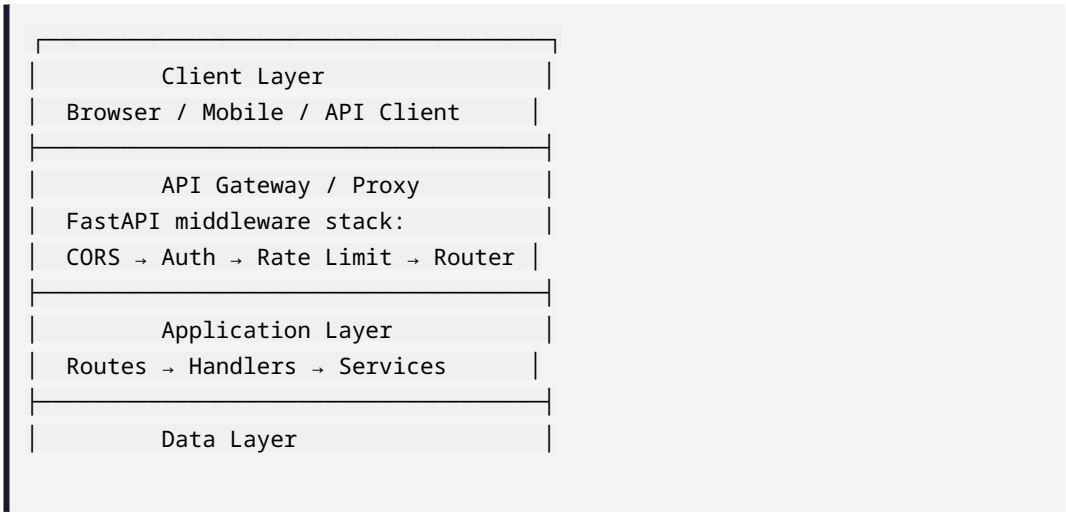
Observer pattern for file changes, Manager pattern for services

Scalability

Single-instance daemon per machine

System Architecture

Layer Architecture



```
│  SQLAlchemy ORM → SQLite/PostgreSQL│
└────────────────────────────────────┘
```

Component Interaction

```
Portfolio Daemon Component Architecture
├── API Layer (FastAPI)
├── Service Layer (Business Logic)
├── Data Layer (SQLAlchemy + SQLAlchemy + SQLite)
└── Client Layer (None)
```

Request Lifecycle

1. **HTTP Request** arrives at FastAPI
2. **CORS middleware** checks origin headers
3. **Auth dependency** extracts and validates JWT
4. **Rate limiter** checks request count per IP
5. **Router** matches path to handler
6. **Handler** executes business logic
7. **Database session** commits or rolls back
8. **Response** returned to client

Dependency Graph

```
 Portfolio Daemon Dependency Flow:
Client → FastAPI → Router → Handler → SQLAlchemy + SQLite → Response
```

Design Patterns Used

Pattern	Location	Purpose
Dependency Injection	FastAPI Depends()	DB sessions, auth
Repository Pattern	SQLAlchemy models	Data abstraction
Middleware Chain	FastAPI middleware	Cross-cutting concerns
Router Module	APIRouter include	Separation of concerns